System-Level Modeling of Multi-Accelerator Architectures via Analytical Cost Model Integration

COMS E6868 - Embedded Scalable Platforms - Spring 2025

Kevin Liu
kl3755@columbia.edu

ABSTRACT

Modern deep learning workloads run on platforms with multiple accelerators connected by a Network-on-Chip (NoC) and shared DRAM. Analytical cost models such as MAESTRO and Timeloop estimate performance for a single accelerator but do not capture system-level effects (inter-accelerator communication, DRAM contention). This project proposes a system-level modeling framework that integrates these accelerator-level models with explicit NoC and memory modeling. At midterm we have (1) implemented output-stationary (ShiDianNao), weight-stationary (NVDLA), and row-stationary (Eyeriss) dataflows in both MAESTRO and Timeloop on the same conv layer (107M MACs) and 256-PE setup, (2) established a reproducible pipeline (Docker, parsing, comparison plots), and (3) compared latency, energy, and EDP across frameworks. Next we will design the system-level framework and integrate NoC/DRAM modeling, then simulate representative DNN workloads on multi-accelerator systems.

1 INTRODUCTION

This report summarizes progress on the project proposed at the start of the semester. The goal is to build a *system-level* modeling framework that reuses analytical accelerator cost models (MAESTRO, Timeloop) and adds explicit modeling of NoC and shared DRAM so that multi-accelerator systems can be evaluated without cycle-accurate simulation.

1.1 Motivation

Different kinds of accelerators are good at different dataflows (e.g., output-stationary vs. weight-stationary vs. row-stationary), but building a real accelerator is time-consuming and costly. It is therefore important to model performance before committing to hardware. Analytical models such as MAESTRO [1] and Timeloop [2] are effective for exploring dataflows and mappings on a single accelerator. They do not, however, model performance degradation from NoC congestion or DRAM contention when multiple accelerators run concurrently. System-level frameworks like SCAR [3] and MAGMA [4] address scheduling and mapping across multiple cores but use their own performance estimators. This project unifies accelerator-level models (MAESTRO, Timeloop) with system-level NoC/DRAM modeling so that we can plug in established cost models and still capture system effects.

2 PROGRESS AND EXPERIMENTAL SETUP

2.1 Accelerator-Level Comparison (Milestones 2 & 3)

We implemented the three representative dataflows—output-stationary (OS, ShiDianNao), weight-stationary (WS, NVDLA), and row-stationary (RS, Eyeriss)—in both MAESTRO and Timeloop and compared their predicted performance on the *same* conv layer.

Workload. One 2D conv layer: batch $N = 1$, input/output channels $C = M = 64$, filter $R = S = 3$, output spatial $P = Q = 56$. Total MACs = $N \cdot M \cdot C \cdot R \cdot S \cdot P \cdot Q = 107{,}495{,}424$ (107M).

Accelerator size. 256 PEs in both tools: MAESTRO via `num_pes: 256` in the HW file; Timeloop via a 2-level arch (1-PE) run with latency scaled by 1/256 to obtain a 256-PE equivalent for fair comparison.

MAESTRO. Hardware described in a `.m` file (PE count, L1/L2 size constraints, NoC and off-chip bandwidth). Dataflow in separate mapping files (TemporalMap, SpatialMap, Cluster over dimensions K, C, R, S, Y, X). We run OS, WS, RS mappings and parse stdout for runtime (cycles) and total energy (in "MAC energy" units). Energy is converted to μJ using 0.21 pJ/MAC.

Timeloop. Architecture and mapping in YAML: hierarchy (MainMemory → Buffer → MACC), technology, `global_cycle_seconds`, and (for a 256-PE variant) spatial mesh. We use the v0.4 front-end and run `timeloop model` for OS, WS, RS with the same problem and 2-level arch inside Docker (`timeloopaccelergy/timeloop-a`). Stats (cycles, energy in μJ, utilization) are parsed into CSV and merged with MAESTRO for plots.

Metrics. We compare latency (cycles), energy (μJ), and EDP (cycles × μJ). Because the two tools use different cycle and energy models, raw latency and energy can differ; EDP is used as the main cross-framework metric. Plots are generated for latency-by-dataflow, energy-by-dataflow, and EDP, with

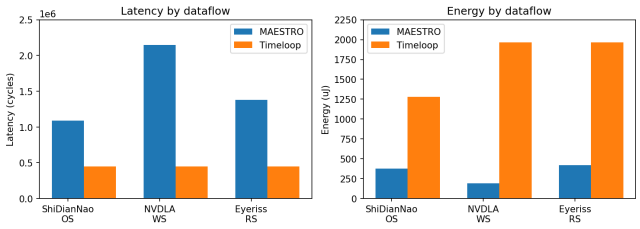

Figure 1: Latency (cycles) and energy (μJ) by dataflow for MAESTRO and Timeloop on the same 107M-MAC layer (256-PE equivalent).

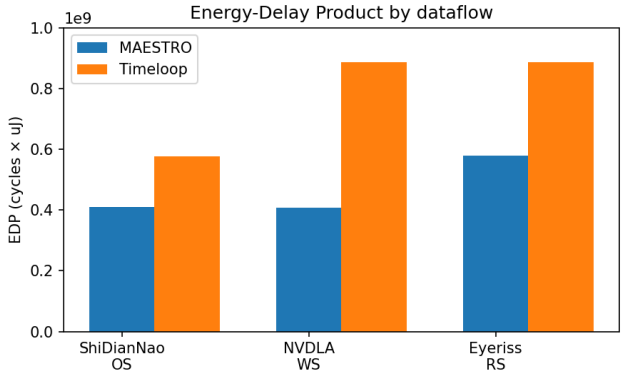

Figure 2: Energy-Delay Product by dataflow. EDP is used as the main cross-framework comparison metric.

fixed Y-axis scales for readability. Figure 1 shows latency and energy by dataflow; Figure 2 shows EDP.

2.2 Reproducibility

The pipeline is automated: (1) `./compare/timeloop/run_timeloop.sh` runs Timeloop OS/WS/RS in Docker and writes `timeloop_*_*.stats.txt` into `compare/results/`; (2) `parse_timeloop.py` and `parse_maestro.py` produce `results_timeloop.csv` and `results_maestro.csv`; (3) `plot_results.py` merges them, applies 256-PE scaling for Timeloop, and outputs `summary_table.csv` and comparison PNGs.

3 VALIDATION PLAN

Accelerator-level (current). Consistency between MAESTRO and Timeloop is assessed by (1) using the same layer and same effective PE count, (2) comparing EDP and relative ranking of dataflows (OS vs WS vs RS), and (3) documenting differences in cycle and energy models (see project `TASKS_GUIDE.md` and midterm slides).

System-level (next). After integrating NoC and DRAM models, we will (1) compare predicted latency and scalability trends with published system-level approaches (e.g., MAGMA, SCAR) where possible; (2) vary the number of accelerator instances and NoC/DRAM bandwidth and report sensitivity; (3) run representative DNN workloads (e.g., a subset of layers from a small CNN) on a multi-accelerator configuration and report end-to-end latency. The goal is trend accuracy for design-space exploration, not cycle-level agreement.

4 NEXT STEPS

(1) **Design the system-level framework** (by Mar. 4): Define the abstraction that takes per-accelerator costs (from MAESTRO/Timeloop) and combines them with NoC and DRAM models.
(2) **Integrate inter-accelerator communication and DRAM modeling** (by Mar. 10): Add analytical or simplified models of NoC traversal and shared DRAM bandwidth allocation.
(3) **Simulation of representative DNN workloads** (by Apr. 15): Run one or more small DNNs (or layer subsets) on a multi-accelerator system in the framework and report latency and scalability.
(4) **System-level optimization** (by May 4): Explore mapping and scheduling strategies inspired by MAGMA and SCAR.
(5) **Final report and presentation** (May 14).

5 MILESTONES (FROM PROPOSAL)

Literature review (Feb. 18); implement OS/WS/RS in MAESTRO and Timeloop and compare (Feb. 25); study dataflow semantics (Feb. 25); design system-level framework (Mar. 4); integrate NoC/DRAM (Mar. 10); midterm presentation and report (Mar. 11) — **done**; simulate DNN workloads on multi-accelerator systems (Apr. 15); system-level optimization (May 4); final analysis and report (May 6); final presentation and report (May 14).

REFERENCES

[1] Hyoukjun Kwon, Prasanth Chatarasi, Michael Pellauer, Angshuman Parashar, Vivek Sarkar, and Tushar Krishna. Understanding reuse, performance, and hardware cost of dnn dataflows: A data-centric approach. In *Proceedings of the 52nd Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*, pages 754–768. ACM, 2019.

[2] Angshuman Parashar, Priyanka Raina, Yakun Sophia Shao, Yu-Hsin Chen, Victor A. Ying, Anurag Mukkara, Rangharajan Venkatesan, Brucek Khailany, Stephen W. Keckler, and Joel Emer. Timeloop: A systematic approach to dnn accelerator evaluation. In *2019 IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS)*, pages 304–315, 2019.

[3] Mohanad Odema, Luke Chen, Hyoukjun Kwon, and Mohammad Abdullah Al Faruque. Scar: Scheduling multi-model ai workloads on heterogeneous multi-chiplet module accelerators. In *2024 57th IEEE/ACM International Symposium on Microarchitecture (MICRO)*, pages 565–579, 2024.

[4] Sheng-Chun Kao and Tushar Krishna. Magma: An optimization framework for mapping multiple dnns on multiple accelerator cores. In *2022 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*, 2022.